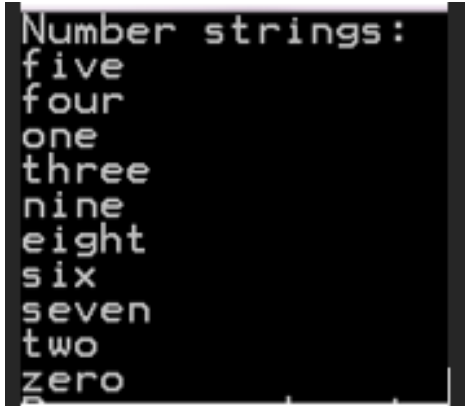


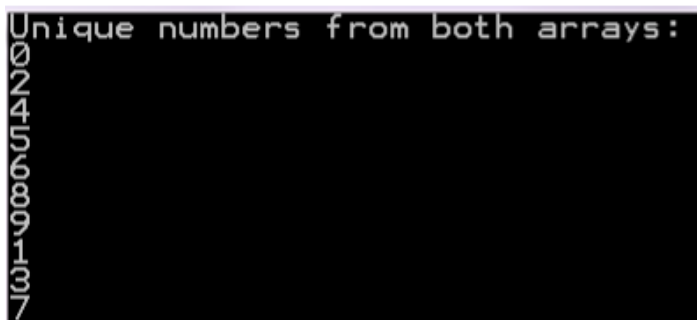
- Produce a sequence of strings representing the text version of a sequence of integers
`int[] numbers = { 5, 4, 1, 3, 9, 8, 6, 7, 2, 0 }`
`string[] strings = { "zero", "one", "two", "three", "four", "five", "six", "seven", "eight", "nine" }`
 Expected output:



```
Number strings:
five
four
one
three
nine
eight
six
seven
two
zero
```

- Create one sequence that contains the unique values from both arrays
`int[] numbersA = { 0, 2, 4, 5, 6, 8, 9 }`
`int[] numbersB = { 1, 3, 5, 7, 8 }`

Expected Output:



```
Unique numbers from both arrays:
0
2
4
5
6
8
9
1
3
7
```

- Snake game is an old computer game. There are many versions, but all involve maneuvering a "snake" around the screen, trying to avoid running the snake into itself or an obstacle. In this task, you have to simulate the game through the application, though it is very simplified version.
 Consider a 10 X 10 board/matrix (refer the following diagram, where every integer value is indicating the cell number). The snake moves Suppose the snake starts to move from cell number 15. The snake can move to 4 possible directions: north (N), east (E), south (S) and west (W). If the snake moves to N, then it actually moves to cell number 5. If it moves to S, then it actually moves to cell number 25. Similarly, if the snake moves to E or W, which means snake is moving to cell number 26 or 24 respectively. So, with every move, the snake covers a cell and consequentially it leaves a trail along the path.

Now consider a sequence of moves, NEEESSWWNNN. That means the snake first moves from its initial position (cell number 15) to cell number 5, then takes a right turn and covers the cell number 6, 7 and 8. After that it takes a down turn and covers 18 and 28, before it takes a left turn and covers cell number 27 and 26. At last it takes again upward turn and covers cell number 16 first and meets the same trail, i.e., cell number 6. It means the snake has run on to itself. In this case, the snake made 9 successful moves and, on the 10th, move it runs into itself.

Again, consider the sequence of moves, SSEEEEEEEE. That means from starting position (cell number 15, the snake moves downward till cell 35 and then takes a right turn to cover cell number 36 to 40, before it runs out of board. Before running out of board it made 7 moves.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Again, consider sequence of moves: WWSSSEEEESSSS. That means the snake first made a left run from start position (cell number 15) and covered cell number 14 and 13 and then a downward move and continued till cell number 43. Then it took a right turn to go till cell number 47 and again moved downwards and continued till cell number 87. This time the snake neither did run onto itself nor run out of board. It completed all the 13 moves successfully.

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100



Instruction:

1. Complete the application by writing the logic in a method separate from Main method to return the number of moves as well as the result of the moves (i.e., “SUCCESS” OR “RANOUTOFBOARD” OR “RANONTOITSELF”) for a given sequence of moves in string format start position as integer value. That means

Method Input: Two

- i. Start position as integer value and
- ii. sequence of moves as string value

Method Output:

Number of moves (made successfully or made before it ran out of board or before it ran onto itself) along with the result of the moves (i.e., “SUCCESS” OR “RANOUTOFBOARD” OR “RANONTOITSELF”).

- 2. The actual logic has to be written inside a method separate from Main method, and will be invoked from Main method.
- 3. All inputs and outputs should be part of Main method and the other method should contain only the business logic.
- 4. Exception handling is MUST
- 5. Follow coding best practices, proper naming conventions etc.

Sample Input and Output:

UTC	Input	Output
UTC04_01	53 and SSSSEENNEEEEESS	{13, RANOUTOFBOARD}
UTC04_02	3 and SSSSESSSSWNNNNNNNN	{14, RANONTOITSELF}
UTC04_03	15 and WWSSEEEEESSSS	{13, SUCCESS}